Aufgabe 2: Gießroboter

Teilnahme-ID: 80513

Bearbeiter dieser Aufgabe:
Peter Staykov

11. April 2026

Inhaltsverzeichnis

1 Lösungsidee **1**

2 Umsetzung **1**
2.1 K-Means-Clustering 2
2.2 Routenoptimierung im Cluster 2
2.3 Bestimmung der optimalen Cluster-Anzahl 2

3 Werkzeuge **2**

4 Beispiele **2**

5 Quellcode (Wesentliche Algorithmen) **6**

1 Lösungsidee

Um das Problem der effizienten Routenplanung für mehrere Roboter zu lösen, unterteile ich die Aufgabe in zwei zentrale Teilprobleme: die Gruppierung der Bäume und die Optimierung der Wege innerhalb dieser Gruppen.

Da mehrere Roboter eingesetzt werden, müssen die Bäume zunächst sinnvoll aufgeteilt werden. Hierfür verwende ich das K-Means-Clustering. Jeder Roboter übernimmt dabei ein Cluster. Die Anzahl der Cluster n entspricht der Anzahl der eingesetzten Roboter. Dies basiert auf der Annahme, dass räumlich nah beieinander stehende Bäume kürzere Wege untereinander aufweisen.

Innerhalb jedes Clusters muss der kürzeste Weg gefunden werden, der alle Bäume verbindet. Ich kombiniere zwei Heuristiken: den Nearest-Neighbor-Algorithmus zur Erstellung einer soliden Startroute und die 2-Opt-Heuristik, um die initiale Route iterativ durch das Auflösen von Überschneidungen zu verbessern.

Da die benötigte Anzahl an Robotern initial unbekannt ist, implementiere ich ein iteratives Verfahren. Ich beginne mit einer kleinen Anzahl an Robotern und erhöhe die Anzahl der Cluster, bis die maximale Weglänge $L_{\max}$ unterschritten wird. Um diesen Prozess zu beschleunigen, nutze ich eine lineare Interpolation zur Schätzung der nächsten Cluster-Anzahl und verfeinere das Ergebnis abschließend durch eine binäre Suche.

Da jedem Roboter eine eigene Station zugeordnet ist, würde jede Platzierung der Ladestation außerhalb der berechneten Route die Gesamtlänge des Pfades zwangsläufig erhöhen. Daher wird diese, ohne Beschränkung der Allgemeinheit, exakt auf die Koordinaten des ersten Baums innerhalb des jeweiligen Clusters gesetzt.

2 Umsetzung

Die Berechnung der Gesamtroute erfolgt durch folgende algorithmische Schritte:

2.1 K-Means-Clustering

Der Algorithmus funktioniert nach folgendem Prinzip in einem euklidischen Raum:

1. Es werden n Centroids (Zentren) zufällig im Raum platziert.
2. Jeder Baum wird dem Centroid zugeordnet, zu dem er die geringste Distanz hat.
3. Die Position jedes Centroids wird zum geometrischen Schwerpunkt aller ihm zugeordneten Bäume verschoben.
4. Diese Schritte werden wiederholt, bis die Positionen der Centroids stabil bleiben.

2.2 Routenoptimierung im Cluster

Nearest Neighbor: Der Algorithmus startet bei einem Baum und wählt als nächsten Punkt stets den Baum aus, der die geringste Distanz zum aktuellen Standort aufweist.

2-Opt-Heuristik: Um die Greedy-Lösung zu verbessern, prüft die 2-Opt-Heuristik systematisch Kantenpaare. Werden zwei Kanten gefunden, deren Überkreuz-Ersetzung die Gesamtlänge reduziert, wird diese Änderung übernommen. Dies geschieht, bis keine Verbesserungen mehr möglich sind.

2.3 Bestimmung der optimalen Cluster-Anzahl

Um die minimale Anzahl k an Robotern effizient zu finden, schätze ich zunächst die Steigung a zwischen zwei Messungen $(k_{\text{prev}}, d_{\text{prev}})$ und $(k_{\text{curr}}, d_{\text{curr}})$:

$$a = \frac{d_{\text{curr}} - d_{\text{prev}}}{k_{\text{curr}} - k_{\text{prev}}}$$

Daraus ergibt sich die nächste zu prüfende Cluster-Anzahl k_{new}:

$$k_{\text{new}} = k_{\text{curr}} + \frac{L_{\max} - d_{\text{curr}}}{a}$$

Sobald ein Intervall gefunden wurde, in dem die maximale Länge unterschritten wird, verwende ich eine binäre Suche auf dem Bereich $[k_{\text{start}}, k_{\text{curr}}]$, um das kleinste k^* zu finden. *Anmerkung:* Für die binäre Suche wird angenommen, dass die maximale Weglänge mit steigendem k monoton fällt. Auch wenn dies durch den heuristischen Ansatz des Clusterings nicht mathematisch streng garantiert ist, liefert die Methode in der Praxis sehr effizient gute Ergebnisse.

3 Werkzeuge

Die Implementierung erfolgte in Python unter Verwendung der Standardbibliotheken `math` (für Geometrie-Berechnungen) und `numpy` (für effiziente Vektoroperationen im K-Means-Clustering).

Bei der Bearbeitung habe ich die Gesamtaufgabe in logische Unterprobleme zerlegt. Zunächst wurde die Optimierung der individuellen Wege der Roboter adressiert. Hierbei wurde intuitiv ein Nearest-Neighbor-Algorithmus implementiert. Die anschließende Visualisierung verdeutlichte jedoch, dass sich Wege kreuzten, was auf eine suboptimale Route hinwies. Zur Lösung wurde die 2-Opt-Heuristik implementiert. Das zweite Teilproblem, die Zuweisung der Roboter zu Einsatzgebieten, wurde aufgrund meiner Erfahrung im Bereich Machine Learning mittels K-Means-Clustering gelöst.

4 Beispiele

```
13
13
0
110 769
5
0
95 704
3
0
```

```
169 696
8
0
68 753
1
0
154 678
7
0
142 754
12
0
80 677
11
0
156 728
9
0
53 684
13
0
104 743
2
0
124 713
6
0
67 719
4
0
113 675
10
```

Listing 1: roboter_01.txt Output

```
18
18
0
169 696
8
0
53 684
13
0
142 754
12
0
291 638
16
0
80 677
11
0
171 607
15
0
156 728
9
0
67 719
4
0
154 678
7
0
124 713
6
0
113 675
10
0
472 789
18
0
```

```
104 743
2
0
49 607
14
0
95 704
3
0
393 712
17
0
68 753
1
0
110 769
5
```

Listing 2: roboter_02.txt Output

```
27
200
32
509 515
137 149 151
116
224 598
168 186 172 192 187 169 171 178 199 196 190 176 184 185
175
752 237
35 60 47 44 61 62 67 36 59 50 65 54 43 63 56
118
228 359
69 88 96 93 70 82 81 90 71 99 89
66
215 710
1 3 8 7 21 32
149
391 267
104 116 118 126 106 110 115 119 133 123
0
155 716
14
23
463 520
146 162 150
59
191 686
9 33 28 10 18 13 23 12
189
752 242
37 64 45 48 57 66 52 55 68 39 58 53 38 40 49 42
16
437 499
138 152
130
199 659
2 5 30 29 11 16 188 175 179 194 200 173 189 174 177
32
478 494
143 154 164 153
244
453 278
102 125 108 131 112 122 113 107 111 103 134 120 109
95
677 252
41 51 46
0
217 446
79
53
462 486
135 156 158 159 167 141 144
30
```

```
473 531
140 145 157
65
197 694
6 25 26 22 31 34
35
241 640
4 195 198
183
402 245
105 128 114 124 127 121 130 117 132 129
57
160 698
15 20 17 24 27 19
39
502 502
139 166 165
49
476 518
136 147 161 155 142 163 160 148
151
254 387
73 86 83 98 92 94 95 100 74 97 78 84 87 76 101 77
109
179 579
170 182 180 181 193 191 183 197
51
217 376
72 80 75 91 85
```

Listing 3: roboter_05.txt Output

```
32
200
64
228 359
69 88 70 93 96 72
78
509 515
137 139 149 151 165
54
484 513
142 155 164 143 166 154 163
26
191 686
9 19 27
21
242 708
7 21 32
80
208 399
75 80 91 85 79
50
781 256
38 40 49 42 53
39
196 600
169 171 199 178 187 192 172
46
199 659
2 5 16 11 30 23
45
677 252
41 62 67
43
281 385
74 97 94 95 100
19
160 698
15 24
41
253 378
76 87 84 90 78 92 98
13
```

```
188 709
31 34
50
437 499
138 152 150 162
65
462 486
135 156 158 159 167 141 144 153
143
752 237
35 37 64 45 48 57 66 61 44 47 60 36 59 50 65 54 56 63 43
45
215 710
1 26 22 25 6
39
219 694
3 8 10 28 18 13 33
246
453 278
102 125 110 106 126 118 116 123 133 119 115 120 109
29
186 634
173 189 174 177
342
452 211
103 134 111 107 113 122 112 108 131 114 124 130 127 105 128 104 121 117 132 129
58
268 358
71 81 82 99 89
91
179 579
170 182 180 181 193 191 183
96
476 518
136 147 146 140 145 161 160 148 157
11
191 671
12 29
41
206 652
175 179 188 194 200
35
241 640
4 195 198
110
224 598
168 186 184 196 190 176 197 185
42
254 387
73 83 86 101 77
143
742 293
39 58 68 55 52 51 46
30
155 716
14 20 17
```

Listing 4: roboter_06.txt Output

5 Quellcode (Wesentliche Algorithmen)

Die vollständige Lösung umfasst Skripte zum Einlesen und Exportieren der geforderten Dateiformate. Im Folgenden sind die wesentlichen algorithmischen Implementierungen für das Clustering, die Pfadfindung und die Bestimmung der Roboteranzahl aufgeführt.

```python
def kmeans(points, k, n_iter=100):
    coords = np.asarray(points, dtype=float)
    n = coords.shape[0]
    if k <= 0:
        return [[] for _ in range(k)], np.zeros((0, 2))
    if k > n:
```

```
        k = n

    centroids = coords[np.random.choice(coords.shape[0], k, replace=False)]

    for _ in range(n_iter):
        distances = np.sqrt(((coords[:, None, :] - centroids[None, :, :]) ** 2).sum(
    axis=2))
        closest_indices = np.argmin(distances, axis=1)

        new_centroids = []
        for i in range(k):
            cluster_coords = coords[closest_indices == i]
            if len(cluster_coords) > 0:
                new_centroids.append(cluster_coords.mean(axis=0))
            else:
                new_centroids.append(coords[np.random.choice(coords.shape[0])])

        new_centroids = np.array(new_centroids)
        if np.allclose(centroids, new_centroids):
            break
        centroids = new_centroids

    clusters = [[] for _ in range(k)]
    for idx, cluster_id in enumerate(closest_indices):
        clusters[cluster_id].append(int(idx))

    return clusters, centroids


def solve_tsp_indices(indices, coords):
    if not indices:
        return [], 0.0
    if len(indices) == 1:
        return [indices[0]], 0.0

    unvisited = indices.copy()
    path = [unvisited.pop(0)]

    while unvisited:
        last = coords[path[-1]]
        next_idx = min(range(len(unvisited)), key=lambda i: euclidean_dist(last,
    coords[unvisited[i]]))
        path.append(unvisited.pop(next_idx))

    length = 0.0
    for i in range(len(path) - 1):
        length += euclidean_dist(coords[path[i]], coords[path[i + 1]])
    return path, length


def optimize_2opt_indices(path_indices, coords, max_iterations=100):
    if len(path_indices) < 4:
        return path_indices[:]

    best_path = path_indices[:]
    for _ in range(max_iterations):
        improved = False
        for i in range(1, len(best_path) - 2):
            for j in range(i + 1, len(best_path) - 1):
                a = coords[best_path[i - 1]]
                b = coords[best_path[i]]
                c = coords[best_path[j]]
                d = coords[best_path[j + 1]]
                old_dist = euclidean_dist(a, b) + euclidean_dist(c, d)
                new_dist = euclidean_dist(a, c) + euclidean_dist(b, d)

                if new_dist < old_dist:
                    best_path[i:j + 1] = best_path[i:j + 1][::-1]
                    improved = True
        if not improved:
            break
    return best_path

```

```python

def find_optimal_k(nodes, max_len):
    n = len(nodes)
    if n == 0: return 0
    if n == 1: return 1

    k_prev = 3
    d_prev = route_max_len_for_k(k_prev, nodes, max_len)
    if d_prev <= max_len: return k_prev

    k_curr = 4
    d_curr = route_max_len_for_k(k_curr, nodes, max_len)
    max_steps = 50
    steps = 0

    while d_curr > max_len and k_curr < n and steps < max_steps:
        steps += 1
        denom = (k_curr - k_prev)
        a = (d_curr - d_prev) / denom if denom != 0 else 0.0

        if a < 0:
            k_float = k_curr + (max_len - d_curr) / a
            k_new = int(math.ceil(k_float))
        else:
            k_new = min(n, max(k_curr + 1, k_curr * 2))

        k_new = max(k_new, k_curr + 1)
        k_new = min(k_new, n)
        k_prev, d_prev = k_curr, d_curr
        k_curr = k_new
        d_curr = route_max_len_for_k(k_curr, nodes, max_len)

    if d_curr > max_len: return None

    k_start = k_prev
    lo = k_start + 1
    hi = k_curr

    while lo < hi:
        mid = (lo + hi) // 2
        d_mid = route_max_len_for_k(mid, nodes, max_len)
        if d_mid <= max_len:
            hi = mid
        else:
            lo = mid + 1

    return lo
```

Listing 5: Algorithmische Kernfunktionen